LESSON 10 · CHEAT SHEET

Collectors I: Collecting Results

Java 8 → Java 21 · results use the shared course dataset
course "today" = 2026-11-07

Key rules

1. A Collector = **supplier** (new container), **accumulator** (add one), **combiner** (merge two), **finisher**. 8
2. Type or order matters? `toCollection(TreeSet::new)`. Never rely on `toSet()`/HashMap order.
3. `toMap(k, v)` needs **unique keys**; otherwise give a **merge function**. Null values → NPE.
4. 4th argument picks the map: `TreeMap::new`, `LinkedHashMap::new`, `EnumMap`.
5. Money: `reducing(ZERO, mapper, BigDecimal::add)`; never `summingDouble`.
6. `teeing` = two collectors, one pass. 12
7. Top level: prefer stream ops. Collectors shine as **downstream** (L11).

Canonical snippets

1 · Total per key with merge + sorted map

```
accounts.stream().collect(toMap(
    Account::customerId, Account::balance,
    BigDecimal::add, TreeMap::new));
// {C001=15700.50, C002=850.75, C003=163950.25, …}
```

2 · BigDecimal total as a collector

```
payments.stream().collect(
    reducing(ZERO, Payment::amount, BigDecimal::add));
// 4429.84
```

3 · Count and total in one pass

```
teeing(counting(),
       reducing(ZERO, Payment::amount,
                BigDecimal::add),
       CountAndTotal::new)
// 12 payments, total 4429.84
```

Collectors in this lesson

Collector	Gives	Since
`toList / toSet`	List / Set (no guarantees)	8
`toCollection(Supplier)`	your collection type	8
`toUnmodifiableList/Set/Map`	immutable, no nulls	10
`toMap(k, v[, merge[, map]])`	Map	8
`joining(sep, pre, suf)`	String	8
`counting / summingInt / averagingInt`	Long / Integer / Double	8
`summarizingInt/Long/Double`	...SummaryStatistics	8
`minBy / maxBy`	Optional	8
`reducing(id, mapper, op)`	any, e.g. BigDecimal	8
`collectingAndThen / mapping`	wrapped result	8
`filtering / flatMapping`	wrapped result	9
`teeing(c1, c2, merger)`	merger result	12

The toMap trap (real output)

```
toMap(Customer::city, Customer::name)
IllegalStateException: Duplicate key Singapore
 (attempted merging values Alice Tan and Chen Wei)
```

Mini diagram: teeing



Real results from the course code

Call	Result
`summarizingInt(seats)`	count=8 sum=2078 min=160 max=471 avg=259.75
`maxBy(amount)`	P005 1500.00
`filtering(SUCCESS, counting())`	8
`teeing(minBy, maxBy)` balances	-150.25 .. 100000.00

Exercise: per method (toMap + merge)	Result
count	{CARD=5, PAYNOW=4, PAYPAL=3}
total	{CARD=2070.29, PAYNOW=1824.30, PAYPAL=535.25}
SUCCESS only	CARD 3 / 1209.89 · PAYNOW 3 / 1564.30 · PAYPAL 2 / 225.00

Anti-patterns → fixes

Don't	Do
`toMap(city, name)`	`toMap(city, name, merge, TreeMap::new)`
value mapper returning null	filter first, or a default value
assert on `toSet().toString()`	compare contents / TreeSet

Don't	Do
`summingDouble(amount)`	`reducing(ZERO, .., add)`
print `IntSummaryStatistics`	format fields with `Locale.ROOT`
3 maps kept in sync	one record + merge, or `teeing`

Lesson 10 · sticky notes

Cut out, stick on your wall, and read one every morning.

Collector = 4 parts

supplier · accumulator
combiner · finisher

toMap needs unique keys

Else: Duplicate key …
Give a merge function!

Pick the map

TreeMap = sorted
LinkedHashMap = first seen
EnumMap = enum order

Money collector

`reducing(ZERO, mapper, BigDecimal::add)`

toSet has no order

Need order? toCollection(
TreeSet::new / LinkedHashSet::new)

teeing [12]

Two collectors, one pass,
one merger.

filtering / flatMapping [9]

Filter / flatMap INSIDE
a collector (groupingBy!).

Downstream power

counting, summing, mapping…
shine inside groupingBy (L11).